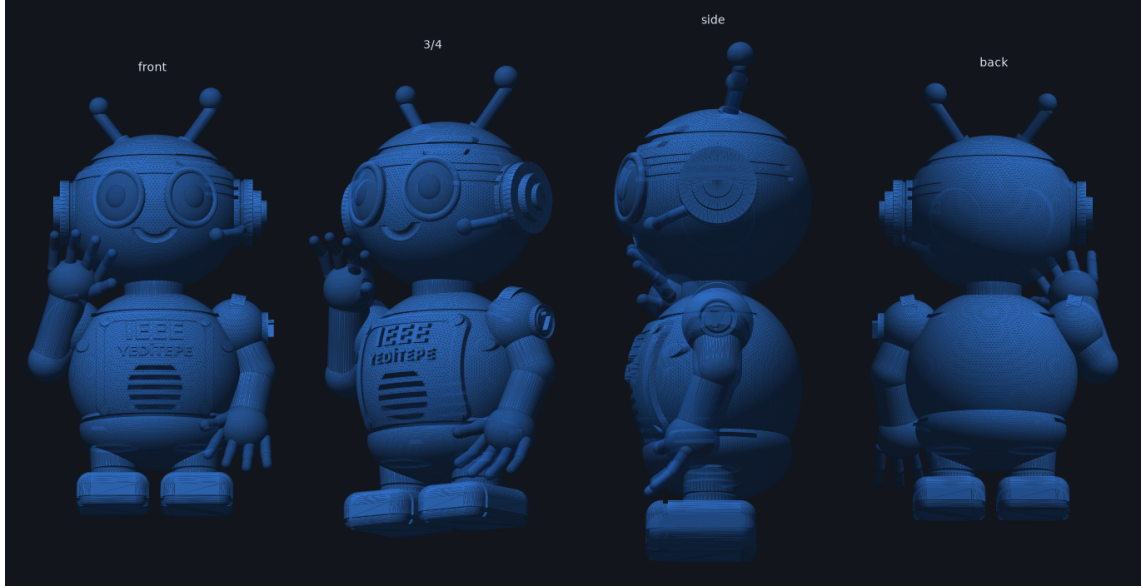


TEPECAN

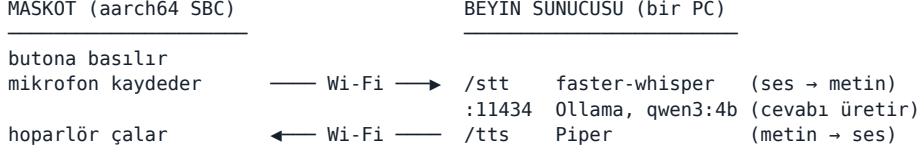
Konuřan maskot — yapım kılavuzu

3D baskı gövde · aarch64 tek kart bilgisayar · yerel yapay zekâ · Türkçe sesli sohbet



1. Ne yapıyoruz

Maskot dinler, sorulanı yapay zekâyâ iletir, cevabı içindeki hoparlörden Türkçe olarak söyler. Ağır iş maskotun içinde değil, ağıdaki bir bilgisayarda çalışır. Maskotun içindeki kartın tek görevi mikrofonu okumak ve hoparlöre yazmak — bu sayede 512 MB RAM'li küçük bir kart yeter ve token maliyeti oluşmaz.



Cevap tamamlanmadan konuşmaya başlar: model ilk cümleyi bitirir bitirmez o cümle seslendirilir, kalanı arka planda üretilir. Algılanan gecikmeyi yarıya indiren tek değişiklik budur.

Verilen kararlar ve gerekçeleri

Karar	Neden
Beyin ayrı bir PC'de	Bu sınıf kartlarda 512 MB - 2 GB RAM var; en küçük dil modeli bile zor sığar, sığsa saniyede 1-3 token üretir. Bir cümle yarım dakika sürerdi.
Büyük kart alınmıyor	85 × 56 mm bir kart maskotun içine girmiyor: kapak açıklığının köşegeni 85.1 mm, o kartın köşegeni 85.8 mm. Kapalı PLA kasada da ısınıyor. Çözdüğü tek darboğaz zaten sunucuya taşındı.
Raspberry Pi yerine muadil	Pi Zero 2 W Türkiye'de bulunamıyor. Kart ince istemci olduğu için marka önemli değil; tek şart 64-bit (aarch64), çünkü openWakeWord'ün istediği onnxruntime yalnız onun için derleniyor.
Metin → ses de sunucuda	Piper'ın Türkçe medium sesi Zero 2 W'de gerçek zamandan yavaş kalabiliyor. Sunucuda çalıştırınca bu risk tamamen kalkar, Pi'ye hazır ses gelir.
Model: Qwen3 4B	Küçük modeller arasında çok dilli tarafı güçlü olanlardan; Türkçesi kullanılabilir. Gemma 3 4B ikinci aday, karşılaştırmaya değer.
Uyandırma kelimesi + buton	"Hey Tepecan" ile butonsuz tetikleme (openWakeWord, yerel). Metin→ses sunucuya taşındığı için Pi'de yer açıldı. Buton yedek olarak duruyor: gürültülü stantta uyandırmayı kapatıp butona dönebilirsin.
Ollama + faster-whisper + Piper	Üçü de ücretsiz ve yerel. Zincirin tamamı internetsiz çalışır, kullanım başına maliyet yoktur.

2. Malzeme listesi

Fiyatlar kabaca, sadece büyüklük fikri versin diye.

#	Parça	Neden bu	≈
1	aarch64 SBC (Orange Pi Zero 2W, Radxa Zero vb.)	Maskotun beyni değil, kulağı ve ağzı: mikrofonu dinler, uyandırma kelimesini yakalar, beyin sunucusuna gönderir. Raspberry Pi Zero 2 W ilk tercihti ama Türkiye'de bulunamıyor. Şart: 64-bit (aarch64) — openWakeWord'ün istediği onnxruntime yalnız bunun için derleniyor. Gövdeye 74.5 × 30, 64.8 × 40 ya da 63.4 × 55 mm'ye kadar kart sığıyor - yani Pi Zero formatı da, Orange Pi Zero 3 gibi daha kare kartlar da.	\$15
2	USB ses kartı (CM108/CM119 yongalı)	Mikrofon girişi + hat çıkışı. Sınıf-uyumlu, yani hiçbir kartta sürücü istemiyor. ReSpeaker HAT'in yerini tutuyor: HAT yalnız Raspberry Pi'de çalışıyor (seeed-voicecard device-tree overlay'i).	\$4
3	Elektret mikrofon kapsülü (3.5 mm fişli)	Ses kartının mic girişine takılır. Kapaktaki havalandırma yarıkları (3 × 37 × 4 mm = 449 mm ²) sesi geçirmeye yetiyor, ayrı port gerekmiyor.	\$2
4	PAM8403 amfi modülü	Ses kartının hat çıkışı hoparlörü süremez. 2 × 3 W D-sınıfı, 5 V'u kartın header'ından alıyor.	\$2
5	Ø30 mm 4 Ω 3 W yuvarlak hoparlör	Gövdedeki yuva bu ölçüye göre: ızgara Ø30 mm, oturma omzu Ø34.8 mm, en fazla 8 mm gövde derinliği. Ø40 mm sığmıyor — göğüs küresel olduğu için o ayak izinde oturma omzu kartın önüne giriyor.	\$3
6	microSD 32 GB, A1 sınıfı	İşletim sistemi. A1 sınıfı olmayan kartlar kartı belirgin yavaşlatır.	\$6
7	5 V 3 A adaptör + karta uyan kablo	Kart + amfi tepe akımı ~800 mA. Zayıf adaptör ses bozulmasına yol açar. Kartın girişi USB-C mi micro-USB mi, sipariş ederken bak.	\$8
8	14 × 14 mm alüminyum soğutucu	Kapalı PLA kasada işlemci sıcaklığını düşürür.	\$1
9	M2.5 × 8 mm kendinden kılavuzlu vida (8 adet)	Dördü adaptör plakasını gövde kulelerine, dördü kartı plakanın standoff'larına. Kılavuz delikleri Ø2.1 mm hazır, ara pula gerek yok.	\$3
10	6 × 6 × 4.3 mm tactile switch + ince kablo	Sırt kapağındaki bas-konuş butonu. Uyandırma kelimesi çalışırken de yedek kalıyor. Basılan kapağı (tepecan_buton.stl) sen basıyorsun.	\$1
11	M3 × 8 kendinden kılavuzlu vida (2 adet)	Sırt kapağını tutturmak için. Kasada yuvaları hazır.	\$1
12	Plastik kelepçe (kablo bağı, birkaç adet)	USB ses kartı ve amfi modülünü adaptör plakasının köşelerindeki yuvalara bağlamak için. Modüllere özel yuva yok, bilerek.	\$1
13	PLA filament, mavi (~400 g)	Gövde. Güneşte/araçta kalacaksa PETG tercih et.	\$12
—	Beyin sunucusu: kulüpte zaten olan bir PC	16 GB RAM yeterli. 8-12 GB VRAM'li bir ekran kartı varsa cevaplar üç kat hızlanır.	—

Kartı ararken tek kırmızı çizgi 64-bit: ARMv6 (Raspberry Pi Zero W 1. nesil) ve ARMv7 kartlarda onnxruntime/tflite tekerleği yok, uyandırma kelimesi çalışmaz. Amazon'da 2.000-2.400 TL'ye görünen Pi Zero W / Zero WH'ler bu gruptan, alma. Raspberry Pi Zero 2 W bulunursa hepsini o karşılar; o zaman

ReSpeaker 2-Mics Pi HAT alıp 2, 3 ve 4 numaralı satırları atlayabilirsin, ama HAT'in seeed-voicecard sürücüsü güncel Raspberry Pi OS'ta sorun çıkarabiliyor. USB yolu daha çok modül ama sıfır sürücü işi.

3. Gövde: 3D baskı

Basılacak dosyalar

Dosya	Ne	Adet
tepecan_body.stl	İçi boş gövde. Sırtta açıklık ve kapak oturma kenarı, iki vida boss'u, hoparlör adası, dört genel amaçlı kart kulesi, kablo yuvası.	1
tepecan_lid.stl	Sırt kapağı. Havalandırma yarıkları ve havşalı vida delikleriyle, baskı pozisyonunda.	1
tepecan_buton.stl	Kapaktaki butonun kapağı: Ø10 mm başlık, 5 mm sap. Yatık basılır, destek istemez.	1
tepecan_plaka.stl	Kart adaptör plakası: altı gövde kulelerine oturur, üstünde kartın kendi delik deseninde standoff'lar var. 66 × 36 × 6 mm, ~20 dakika. Kart değişirse yalnız bu yeniden basılır.	1
tepecan_solid.stl	İçi dolu vitrin figürü. Elektronik koymayacaksan bunu bas, diğerlerini atla.	0-1

Baskı ayarları

Ayar	Değer	Not
Malzeme	PLA / PLA+	PETG daha ısıya dayanıklı
Katman	0.20 mm	Yüz için 0.16 daha temiz
Duvar	3 perimetre	
Dolgu	%10-15 gövde, %20 kapak	
Destek	Evet, ağaç/tree, her yerden	Çene altı, kollar, eller, kulaklık diskleri, mikrofon kolu, antenler
Brim	5 mm	Antenler için
Yönlendirme	Figür ayakta	Taban zaten düzleştirilmiş

Ölçüler: 240 mm boy (anten uçları dahil), 138 mm en, 94 mm derinlik. **Z eksenini en az 250 mm olan bir yazıcı gerekiyor** — Ender 3, P1S, X1C uygun; A1, A1 mini, MK4 gibi 180-210 mm'lik yazıcılara sığmaz. Kabaca 300-400 g filament ve 30-45 saat. Daha kısa baskı için modeli --height 180 ile yeniden üretebilirsin; cidar 3 mm ve vidalar M3 olarak kalır.

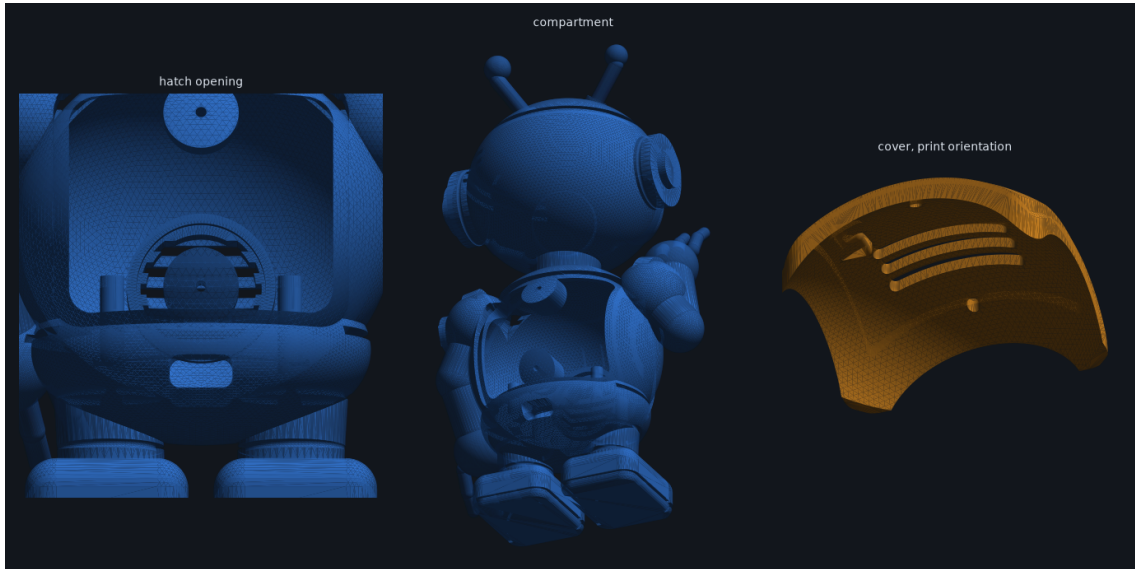
Gövdedeki elektronik detayları

Bunların hepsi modelde hazır; ölçüler govde/dogru.py ile her baskı öncesi ölçülüyor.

- **Göğüs ızgarası** — Ø30 mm dairesel alan içinde 5 yatay yarık, 2.7 mm yükseklik, merkezi tabandan 65 mm'de. YEDİTEPE yazısına 3.2 mm pay var.
- **Hoparlör yuvası** — ızgaranın arkasında düz omuzlu bir ada: göğüs duvarı küresel olduğu için düz yüzü bir hoparlör ona yaslanamıyor, ada sarkmayı dolduruyor. Cep Ø32 mm, omuz 2.4 mm. Ø40 mm hoparlör bu yüzden sığmıyor: o ayak izinde omuz y = 11.9 mm'ye kadar geriliyor ve kartın önüne giriyor.
- **Kart montaj kuleleri** — Ø6 mm, dört adet, **karttan bağımsız** 44 × 28 mm aralıkta. Üst yüzeyleri tabandan 63.8 mm'de, tepelerinde M2.5 için Ø2.1 mm kılavuz delik, 8 mm derin. Kartın kendi delik deseni bu kulelerde değil, üstlerine vidalanan adaptör plakasında.

- **Adaptör plakası** — 66 × 36 × 6 mm ayrı parça. Gövdeyi kart seçiminden kurtarıyor: kart değişirse 30-45 saatlik gövde değil, 20 dakikalık plaka yeniden basılıyor. Köşelerindeki kelepçe yuvalarına USB ses kartı ve amfi modülü bağlanıyor.
- **Kapak açıklığı** — dıştan 73 × 62 mm, oturma kenarından geçiş 65 × 54 mm, köşegen 85 mm. Genişliği kart değil, kartı vidalayan tornavida belirledi: plakadaki vidalar $x = \pm 29$ mm'de ve açıklığın alt köşesine yakın; bu ölçüde kenara 3.7 mm kalıyor, tornavida rahat giriyor. Plaka açıklıktan yan yatırılarak geçiyor (köşegen 75 < 85 mm).
- **Vida boss'ları** — Ø18.7 mm, kavite yüzeyinden 12 mm içeri; M3 vidaya kapak eti ile birlikte 15 mm dış kalıyor.
- **Buton yuvası** — sırt kapağının iç yüzünde: 6.6 mm kare cep, 4.6 mm derin, önünde 2 mm sap kılavuzu, dışa Ø4.2 mm delik.
- **Kablo yuvası** — alt sırtta 15.6 × 7.8 mm, tabandan 38 mm'de; micro-USB fişi kıvrılmadan geçiyor.

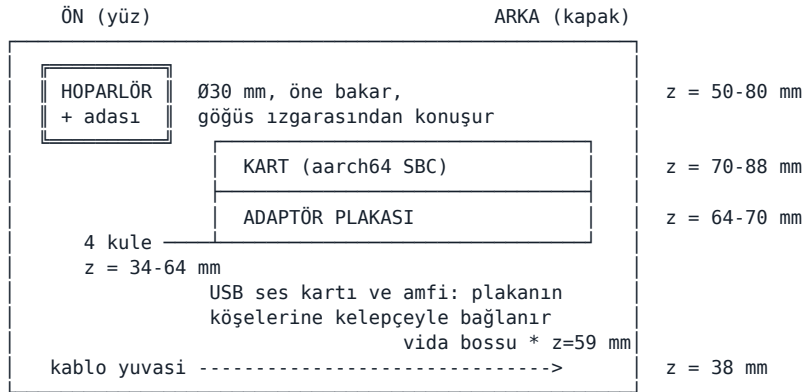
Işıklı gözler denenmedi: RGB LED'leri boyundan geçen bir ışık kanalıyla gözlere taşımak, gözleri şeffaf filamentten ayrı basmayı ve kafayı iki parça yapmayı gerektiriyor. Maskotu en canlı gösterecek şey bu, ama yapımı belirgin zorlaştırdığı için modele girmedim.



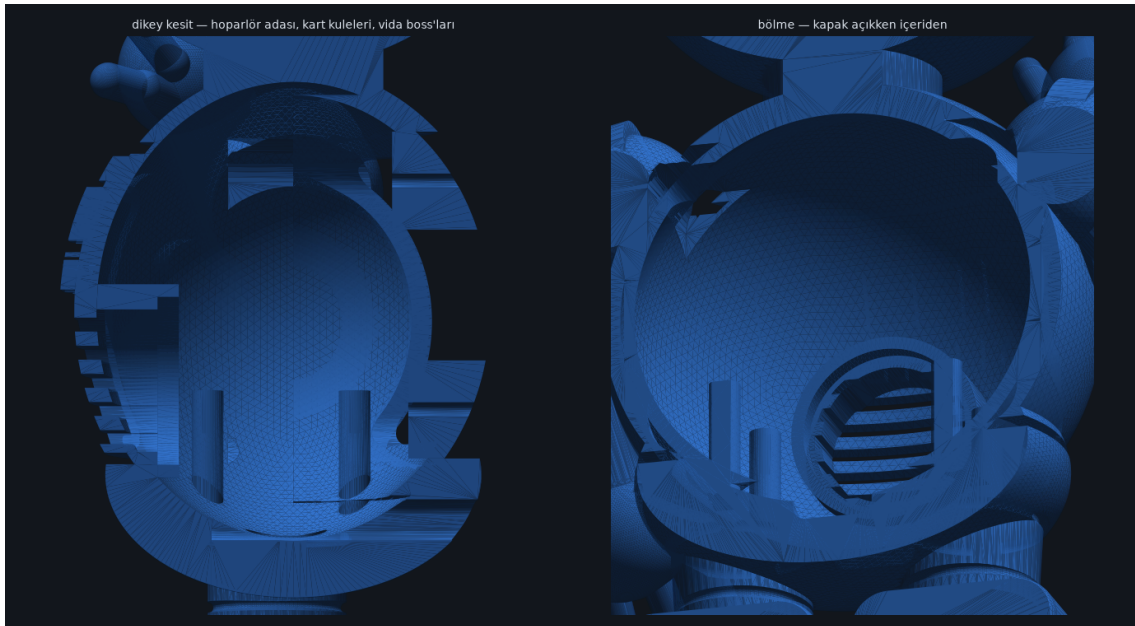
Soldan: sırt açıklığı ve kapak oturma kenarı; bölmenin dıştan görünüşü; kapak, baskı pozisyonunda (dış yüz yukarı, destek gerekmez).

4. İç yerleşim ve montaj

İç boşluk elipsoid olduğu için kutu ölçüsünden cömert: en geniş yerinde 90 × 75 mm, toplam yükseklik 87 mm. Boşluk tabandan 34 mm'de başlıyor, 121 mm'de bitiyor. Aşağıdaki yükseklikler tabandan ölçülü.



Kart, alt vida bossunun (z = 49.8-68.5 mm) tepesinin üstünde duruyor; bu yüzden arka sınırı boss değil kavite duvarı belirliyor. Sığan en büyük kart: 30 mm derinse 74.5 mm, 40 mm derinse 64.8 mm, 50-55 mm derinse 63.4 mm genişlik (derin kartlar plakada 7-10 mm geriye kaydırılıyor). Plakanın üstünde 40 mm yükseklik var. `dogrula.py` bu sayıları her baskı öncesi ölçüyor.



Solda gövdenin dikey kesiti: kart kuleleri ve hoparlör adası. Sağda kapak açırken bölmenin içi.

Montaj sırası

Gövde baskısı karta bağlı değil: plakayı kart gelmeden de basabilirsin, ama delik desenini kartın kendisinden kumpasla ölçüp üretmek en güvenlisi.

- Gövdeyi ve kapağı bas, destekleri temizle. Kapağın açıklığa boşlukla oturduğunu kuru kuruya dene.
- Kart elindeyken montaj deliklerinin aralığını kumpasla ölç. `tepecan_model.py` içinde `BOARD_HOLE_X` / `BOARD_HOLE_Y` değerlerini gir (yarı ölçü: 58 × 23 mm delik için 58/2 ve 23/2), yeniden üret, `tepecan_plaka.stl`'i bas. Kart 35 mm'den derinse `BOARD_OFFSET_Y` ile geriye kaydır: 50 mm için -7, 55 mm için -10 mm.

- Hoparlörü göğüs ızgarasının arkasındaki adaya, düz omza yüzü öne bakacak şekilde otur ve kenarından yapıştır. **Arkasını olabildiğince kapalı tut** — omzun çevresini silikonla sızdırmaz yapmak ses kalitesini gözle görülür artırır, bedava kazançtır.
- İşlemcinin üstüne soğutucuyu yapıştır (kartı plakaya vidalamadan önce yap, sonra elin girmez).
- Kartı plakanın üstündeki dört standoff'a M2.5 vidalarla sabitle. USB ses kartını ve PAM8403'ü plakanın köşe yuvalarına kelepçeyle bağla, kabloları kısa kes. Bu işi gövdenin dışında yap — plaka avucunda dururken çok rahat.
- Hazır plakayı açıklıktan **yan yatırarak** içeri sok (düz geçmez, köşegen 75 mm) ve dört M2.5 vidayla gövde kulelerine sabitle. Kartın güç girişi kablo yuvasına baksın.
- Hoparlör kablosunu PAM8403'ün çıkışına, mikrofonu ses kartının mic girişine tak.
- Buton kapağını (tepecan_buton.stl) sırt kapağının **dışından** deliğe geçir. Switch'i kapağın iç yüzündeki cebe, pistonu kapağa bakacak şekilde bastır; sap tam pistonu dayandır. İki telini kartın bir GPIO hattına ve GND'ye bağla. Telleri kapağı kapatırken sıkışmayacak kadar uzun bırak.
- Güç kablosunu alt sırttaki yuvadan geçir, karta tak.
- Kapağı iki M3 vidayla kapat. Vidalar havşalı deliklere oturur, başları yüzeyle aynı hizada kalır.

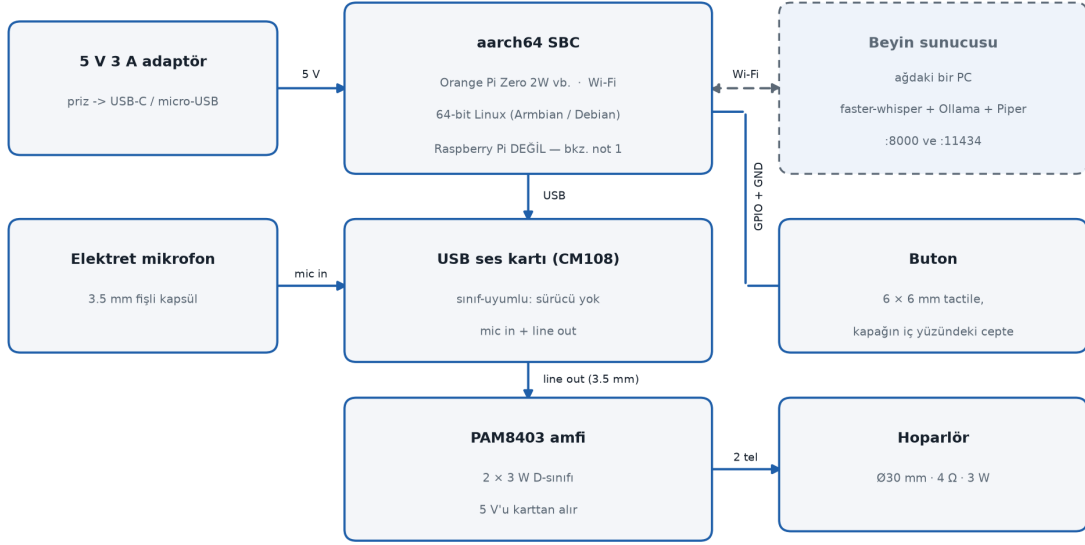
Kablolama

Nereden	Nereye	Not
Elektret mikrofon	USB ses kartı, mic girişi	3.5 mm fiş
USB ses kartı, hat çıkışı	PAM8403 girişi	3.5 mm fiş ya da iki tel
PAM8403 çıkışı	Hoparlör (+/-)	Kutup önemli değil, tek hoparlör var
PAM8403 besleme	Kartın 5 V ve GND pini	Amfi 5 V ister, 3.3 V değil
USB ses kartı	Kartın USB portu	USB-C ise araya adaptör gerekir
Buton bacak 1	Bir GPIO hattı	Kodda TEPECAN_BUTTON_PIN / TEPECAN_BUTTON_CHIP
Buton bacak 2	GND	
5 V adaptör	Kartın güç girişi	Varsa veri portuna değil, güç portuna

Ses cihazını seçmeyi unutma: USB ses kartı çoğu kartta ALSA'nın varsayılanı olmuyor. python3 -m sounddevice cihazları listeler, sonra TEPECAN_MIC ve TEPECAN_SPK ortam değişkenlerine indeksi ya da adın bir parçasını yaz.

Tepecan — bağlantı şeması

Maskotun içindeki tüm kablolama. Beyin sunucusu ağ üzerinden bağlanır, kablolu yoktur.



Notlar

1. ReSpeaker HAT yalnız Raspberry Pi'de çalışır (seeed-voicecard overlay'i); muadil kartta USB ses kartı kullanılıyor.
2. Kart 64-bit olmalı: openWakeWord'ün istediği onnxruntime yalnız aarch64 için derleniyor. ARMv6/ARMv7 kart alma.
3. Adaptör 3 A olmalı; zayıf besleme sesle cızırtı yapar.
4. Butonun kutbu yok, iki bacağı GPIO ve GND'ye gider.
5. Ses cihazını TEPECAN_MIC / TEPECAN_SPK ile seç;
``python3 -m sounddevice`` cihazları listeler.

Bağlantı şeması.

Isı: kapalı PLA kasada bu sınıf bir kart boşta 50-55 °C civarında kalır. Kapaktaki üç yarık ve kablo yuvası hafif hava akışı sağlıyor, soğutucuyla birlikte sorun çıkarmaz. PLA 60 °C dolayında yumuşamaya başlar; maskot güneşte veya araçta kalacaksa gövdeyi PETG bas.

5. Beyin sunucusu kurulumu

Kulüpteki herhangi bir PC. Linux varsayılıyor; Windows'ta WSL2 içinde de çalışır.

Ollama ve model

```
curl -fsSL https://ollama.com/install.sh | sh

ollama pull qwen3:4b          # ana aday
ollama pull gemma3:4b        # ikinci aday, karşılaştırmak için

# Ağdan erişilebilir olsun diye:
OLLAMA_HOST=0.0.0.0 ollama serve
```

Piper (Türkçe ses)

```
mkdir -p ~/piper && cd ~/piper
# piper çalıştırılabilir dosyasını ve şu ikisini buraya indir:
#   tr_TR-dfki-medium.onnx
#   tr_TR-dfki-medium.onnx.json

echo "Merhaba, ben Tepecan." | ./piper --model tr_TR-dfki-medium.onnx --output_file test.wav
aplay test.wav                # sesi duyuyorsan Piper hazır
```

Ses servisi

```
pip install -r requirements-sunucu.txt
uvicorn beyin:app --host 0.0.0.0 --port 8000

# GPU varsa belirgin daha doğru:
STT_DEVICE=cuda STT_MODEL=medium uvicorn beyin:app --host 0.0.0.0 --port 8000
```

Sunucunun ağdaki adını tepecan-beyin yaparsan Pi tarafında hiçbir ayar değiştirmen gerekmez. Yapamıyorsan Pi'de TEPECAN_BRAIN=http://192.168.1.42 gibi ver.

Kontrol: başka bir makineden curl http://tepecan-beyin.local:8000/health komutu {"ok": true, ...} döndürmeli.

6. Maskot (Pi) kurulumu

Raspberry Pi OS Lite, 64 bit. Kurulum sırasında Wi-Fi ve SSH'ı aç.

Ses kurulumu

USB ses kartı sınıf-uyumlu, yani sürücü kurulumu yok. Tek iş doğru cihazı seçmek.

```
sudo apt update && sudo apt install -y python3-pip libportaudio2
```

```
# Kart görünüyor mu:
arecord -l          # USB ses kartı listede olmalı
aplay -l
```

```
# Kayıt ve çalma testi:
arecord -d 3 -f cd deneme.wav && aplay deneme.wav
```

```
# Python tarafında hangi indeks olduğunu gör:
python3 -m sounddevice
```

```
# Sonra indeksi (ya da adın bir parçasını) ortam değişkenine yaz:
export TEPECAN_MIC="USB"
export TEPECAN_SPK="USB"
```

Mikrofon sesi kısık gelirse alsamixer ile USB kartını seç (F6) ve Mic ile Capture seviyelerini yükselt; birçok CM108 dongle sıfır kazançla geliyor.

Tepecan

```
mkdir -p ~/tepecan && cd ~/tepecan
# tepecan.py, persona.txt, bilgiler.txt, requirements-pi.txt dosyalarını buraya koy

pip install --break-system-packages -r requirements-pi.txt
python3 tepecan.py
```

"Hey Tepecan" de ya da sırt kapağındaki butona bas; kayıt sen susunca kendiliğinden biter. Terminalde durumu ve konuşulan metni görürsün. Uyandırma kelimesi modelini edinmek için yazılım/maskot/UYANDIRMA.md dosyasına bak — model dosyası yoksa program yine çalışır, sadece buton tetikler.

Açılıştan otomatik başlasın

/etc/systemd/system/tepecan.service dosyasını oluştur:

```
[Unit]
Description=Tepecan sesli asistan
After=network-online.target sound.target
Wants=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/tepecan
Environment=TEPECAN_BRAIN=http://tepecan-beyin.local
ExecStart=/usr/bin/python3 /home/pi/tepecan/tepecan.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl enable --now tepecan
sudo journalctl -u tepecan -f      # canlı günlük
```

7. Ayarlar

Değişken	Varsayılan	Ne işe yarar
TEPECAN_BRAIN	http://tepecan-beyin.local	Beyin sunucusunun adresi
TEPECAN_MODEL	qwen3:4b	Ollama'da kullanılacak model
TEPECAN_LLM_BACKEND	ollama	ollama veya claude
TEPECAN_BUTTON_PIN	17	Butonun bağlı olduğu GPIO
TEPECAN_WAKE_MODEL	~/tepecan/hey_tepecan.onnx	Uyandırma modeli; yoksa sadece buton
TEPECAN_WAKE_THRESHOLD	0.5	Uyandırma güven eşiği (yükselt = az yanlış tetikleme)
TEPECAN_WAKE_HITS	2	Ardışık kaç blok eşiği aşmalı (gürültüde 3 yap)
STT_MODEL	small	Sunucuda: whisper boyutu (small / medium)
STT_DEVICE	cpu	Sunucuda: cpu veya cuda
PIPER_MODEL	~/piper/tr_TR-dfki-medium.onnx	Sunucuda: Türkçe ses modeli

Stant gününde daha kaliteli cevap istersen tek satır:

```
TEPECAN_LLM_BACKEND=claude ANTHROPIC_API_KEY=... python3 tepecan.py
```

Claude arka ucu claude-opus-5 kullanır, kişilik promptunu önbelleğe alır ve sohbet için düşük efor ayarıyla çalışır. Soru başına maliyet kabaca 0,25 TL. Bu yolu asıl arka uç yaparsan sunucu tarafı yedek model (fallbacks) ayarını da eklemek gerekir; şu hâliyle eklenmedi.

8. Küçük modelden iyi cevap almanın yolu

Model 4 milyar parametrelidir. Ondan bilgiyi *bilmesini* değil, verdiğimiz metni *okumasını* istiyoruz. Kaliteyi belirleyen şey model değil, aşağıdakiler:

- **bilgiler.txt dosyasını doldur.** Kulübe dair her gerçek buraya. Her soruda modele veriliyor. Doğru cevabı sağlayan asıl mekanizma bu — en yüksek getirili iş, modeli değiştirmek değil bu dosyayı yazmaktır.
- **2-3 cümle sınırını gevşetme.** persona.txt bunu dayatıyor. Küçük modeller uzadıkça dağılır, üstelik hoparlörden uzun cevap dinlemek sıkıcıdır.
- **Hafızayı artırma.** Altı mesaj tutuluyor, üç dakika sessizlikte sıfırlanıyor. Uzun geçmiş küçük modelde kaliteyi düşürür.
- **repeat_penalty 1.15** kendini tekrarlama döngüsünü kesiyor, **num_predict 160** cevabı kısa tutuyor.
- Qwen3 düşünme modunda <think> etiketi üretebiliyor; hem kapatılıyor hem de metinden temizleniyor. Yoksa hoparlör modelin iç monologunu okur.
- **Model seçimini ölçerek yap:** aynı on soruyu qwen3:4b ve gemma3:4b'ye sor, kulüpten birkaç kişiyle kör karşılaştı. Bir akşamlik iş, kararı bitirir.

9. Test ve sorun giderme

Belirti	Muhtemel sebep	Ne yapmalı
Hiç ses kaydedilmiyor	Yanlış ses cihazı seçili	arecord -l ve python3 -m sounddevice ile USB kartını bul, TEPECAN_MIC'e yaz
Buton çalışmıyor	gpiozero yalnız Raspberry Pi'de çalışır	python-periphery kurulu mu bak; TEPECAN_BUTTON_CHIP ve TEPECAN_BUTTON_PIN'i kartının pin haritasına göre ayarla
Kayıt hemen bitiyor	VAD gürültüyü konuşma sanıyor ya da mikrofon sessiz	alsamixer ile giriş seviyesini yükselt; kodda Vad(2) yerine Vad(1) dene
Kayıt hiç bitmiyor	Ortam gürültülü, sessizlik algılanmıyor	Vad(3) yap; SILENCE_MS değerini 600'e düşür
STT hatası / bağlanamıyor	Sunucu adresi yanlış veya güvenlik duvarı	karttan curl http://tepecan-beyin.local:8000/health dene
Cevap geliyor ama ses yok	Piper modeli sunucuda bulunamıyor	Sunucu günlüğüne bak; PIPER_MODEL yolunu tam ver
Ses cızırtılı, kesik	Adaptör zayıf	5 V 3 A adaptör kullan; powerbank'ten besleme
Model saçmalıyor, konuyu kaçırıyor	bilgiler.txt boş ya da çok uzun	Doldur ama 1500 karakteri geçme
Cevaplar çok yavaş	Sunucuda GPU yok	Daha küçük model (qwen3:1.7b) veya STT_MODEL=base dene
Hoparlörden model düşüncesi okunuyor	<think> etiketi sızmış	Ollama sürümünü güncelle; think:false destekli olmalı
Durmadan kendi kendine uyanıyor	Uyandırma eşiği düşük, ortam gürültülü	TEPECAN_WAKE_THRESHOLD=0.6, sonra TEPECAN_WAKE_HITS=3
Uyandırma kelimesine hiç tepki vermiyor	Eşik yüksek, mikrofon kısık ya da model yolu yanlış	Açılıştaki "Uyandırma kelimesi hazır" satırını gör; eşiği 0.4 yap

Beklenen gecikme

Sunucu	Soru sonundan ilk sese
Ekran kartlı PC (8-12 GB VRAM)	yaklaşık 2-3 saniye
Ekran kartsız PC, qwen3:4b	yaklaşık 5-7 saniye
Ekran kartsız PC, qwen3:1.7b	yaklaşık 3-4 saniye

10. Kodlar

yazilim/sunucu/beyin.py — beyin sunucusu (ses↔metin)

```
#!/usr/bin/env python3
"""
Tepecan beyin sunucusu – ses->metin ve metin->ses.

Bu iki iş de maskotun içindeki Pi için ağır; buraya alındı. LLM ayrı bir
servis (Ollama) olarak aynı makinede çalışır, Pi ona doğrudan bağlanır.

    uvicorn beyin:app --host 0.0.0.0 --port 8000

Ortam değişkenleri:
    STT_MODEL=small          CPU'da makul, Türkçesi iyi (varsayılan)
    STT_DEVICE=cuda          GPU varsa
    PIPER_MODEL=/yol/tr_TR-dfki-medium.onnx
"""

import os
import subprocess
import tempfile

from fastapi import FastAPI, File, UploadFile
from fastapi.responses import Response
from pydantic import BaseModel
from faster_whisper import WhisperModel

STT_MODEL = os.getenv("STT_MODEL", "small")
STT_DEVICE = os.getenv("STT_DEVICE", "cpu")
STT_COMPUTE = os.getenv("STT_COMPUTE", "int8" if STT_DEVICE == "cpu" else "float16")

PIPER_BIN = os.getenv("PIPER_BIN", "piper")
PIPER_MODEL = os.getenv("PIPER_MODEL",
                        os.path.expanduser("~/piper/tr_TR-dfki-medium.onnx"))

print(f"faster-whisper yükleniyor: {STT_MODEL} ({STT_DEVICE}/{STT_COMPUTE})")
whisper = WhisperModel(STT_MODEL, device=STT_DEVICE, compute_type=STT_COMPUTE)
print("Hazır.")

app = FastAPI(title="Tepecan beyin")

@app.get("/health")
def health():
    return {"ok": True, "stt": STT_MODEL, "device": STT_DEVICE}

@app.post("/stt")
async def stt(audio: UploadFile = File(...)):
    """WAV al, Türkçe metin döndür."""
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        tmp.write(await audio.read())
        tmp.flush()
        segments, info = whisper.transcribe(
            tmp.name,
            language="tr",          # dili sabitlemek tahmin adımını atlatır, hızlandırır
            vad_filter=True,
            beam_size=1,           # sohbet için hız > son kırıntı doğruluk
        )
        text = " ".join(s.text.strip() for s in segments).strip()

    print(f"STT [{info.duration:.1f} sn]: {text}")
    return {"text": text}

class TTSIstek(BaseModel):
    text: str

@app.post("/tts")
def tts(istek: TTSIstek):
    """Metin al, WAV döndür. Piper burada çalışır, Pi'de değil."""
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        subprocess.run(
```

```
[PIPER_BIN, "--model", PIPER_MODEL, "--output_file", tmp.name],
input=istek.text.encode("utf-8"),
stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
check=True,
)
tmp.seek(0)
wav = open(tmp.name, "rb").read()

print(f"TTS [{len(wav)/1024:.0f} KB]: {istek.text[:60]}")
return Response(content=wav, media_type="audio/wav")
```

yazilim/maskot/tepecan.py — maskotun içindeki program

```
#!/usr/bin/env python3
"""
Tepecan — maskotun içindeki sesli asistan.

Raspberry Pi Zero 2 W üzerinde çalışır. Pi'nin tek işi mikrofonu okumak ve
hoparlöre yazmak; ses->metin, LLM ve metin->ses beyin sunucusunda.

    "Hey Tepecan" (ya da buton)
        -> kayıt (sen susunca biter) -> /stt -> Ollama -> cümle cümle /tts -> hoparlör

Mikrofon tek bir sürekli akış olarak açılır; uyandırma kelimesi dedektörü ve
kayıt aynı akıştan beslenir. Ayarların hepsi ortam değişkeninden okunur.
"""

import json
import os
import queue
import re
import sys
import threading
import time
import wave
from io import BytesIO

import numpy as np
import requests
import sounddevice as sd
import webrtcvad

# -----
# Ayarlar
# -----
BRAIN = os.getenv("TEPECAN_BRAIN", "http://tepecan-beyin.local")
STT_URL = f"{BRAIN}:8000/stt"
TTS_URL = f"{BRAIN}:8000/tts"
OLLAMA_URL = f"{BRAIN}:11434"
OLLAMA_MODEL = os.getenv("TEPECAN_MODEL", "qwen3:4b")

LLM_BACKEND = os.getenv("TEPECAN_LLM_BACKEND", "ollama") # ollama | claude
CLAUDE_MODEL = "claude-opus-5"

BUTTON_PIN = int(os.getenv("TEPECAN_BUTTON_PIN", "17")) # GPIO hattı
BUTTON_CHIP = os.getenv("TEPECAN_BUTTON_CHIP", "/dev/gpiochip0") # Pi dışı kartlar

def _device(env):
    """TEPECAN_MIC / TEPECAN_SPK: indeks ya da cihaz adının bir parçası.

    USB ses kartı çoğu kartta ALSA'nın varsayılanı olmuyor; boş bırakılırsa
    sounddevice varsayılanı kullanır. `python3 -m sounddevice` ile listelenir.
    """
    v = os.getenv(env, "").strip()
    if not v:
        return None
    return int(v) if v.lstrip("-").isdigit() else v

MIC_DEV = _device("TEPECAN_MIC")
SPK_DEV = _device("TEPECAN_SPK")
WAKE_MODEL = os.getenv("TEPECAN_WAKE_MODEL", # boş -> sadece buton
    os.path.expanduser("~/tepecan/hey_tepecan.onnx"))
WAKE_THRESHOLD = float(os.getenv("TEPECAN_WAKE_THRESHOLD", "0.5"))
WAKE_HITS = int(os.getenv("TEPECAN_WAKE_HITS", "2")) # ardışık kaç blok eşiği aşmalı

MIC_RATE = 16000 # STT, VAD ve uyandırma modeli 16 kHz ister
BLOCK = 1280 # 80 ms — openWakeWord'ün beklediği pencere
VAD_FRAME = 160 # 10 ms — webrtcvad'in kabul ettiği en küçük pencere
VAD_BYTES = VAD_FRAME * 2
VOICED_FRAMES = 3 # 80 ms içinde en az 30 ms konuşma varsa "konuşuyor"
SILENCE_MS = 800 # bu kadar sessizlikte kayıt biter
MAX_RECORD_S = 15 # güvenlik sınırı
MEMORY_TURNS = 6 # kaç mesaj hatırlansın (küçük modelde artırma)
MEMORY_TIMEOUT_S = 180 # bu kadar sessizlikten sonra sohbeti sıfırla
```

```

HERE = os.path.dirname(os.path.abspath(__file__))

def _read(name, fallback=""):
    path = os.path.join(HERE, name)
    if os.path.exists(path):
        with open(path, encoding="utf-8") as f:
            return f.read().strip()
    return fallback

PERSONA = _read("persona.txt", "Sen Tepecan'sın, sevimli bir kulüp maskotusun.")
FACTS = _read("bilgiler.txt")
SYSTEM_PROMPT = PERSONA + ("\n\nBildiklerin:\n" + FACTS) if FACTS else ""

def set_state(state):
    """bekliyor | dinliyor | dusunuyor | konusuyor – LED takarsan buraya bağla."""
    print(f"[{state}]", flush=True)

# -----
# 1) Mikrofon – tek sürekli akış
# -----
vad = webrtcvad.Vad(2)          # 0 gevşek .. 3 agresif

def speaking(block):
    """80 ms'lik bloğu 10 ms'lik parçalara bölüp VAD'e sorar."""
    voiced = sum(vad.is_speech(block[i:i + VAD_BYTES], MIC_RATE)
                  for i in range(0, len(block), VAD_BYTES))
    return voiced >= VOICED_FRAMES

def drain(mic):
    """Biriken sesi at. Tepecan kendi konuşmasını duymasın diye şart."""
    while mic.read_available >= BLOCK:
        mic.read(BLOCK)

class WakeWord:
    """openWakeWord sarmalayıcısı. Model yoksa sessizce devre dışı kalır."""

    def __init__(self, model_path, threshold, needed_hits):
        self.model, self.threshold, self.needed = None, threshold, needed_hits
        self.hits = 0
        if not model_path or not os.path.exists(model_path):
            print(f"Uyandırma modeli yok ({model_path}); sadece buton çalışır.")
            return
        try:
            from openwakeword.model import Model
            self.model = Model(wakeword_models=[model_path], inference_framework="onnx")
            print(f"Uyandırma kelimesi hazır: {os.path.basename(model_path)} "
                  f"(eşik {threshold})")
        except Exception as exc:
            print(f"Uyandırma kelimesi yüklenemedi ({exc}); sadece buton çalışır.")

    @property
    def enabled(self):
        return self.model is not None

    def reset(self):
        self.hits = 0
        if self.model is not None and hasattr(self.model, "reset"):
            self.model.reset()

    def hears(self, block):
        if self.model is None:
            return False
        scores = self.model.predict(np.frombuffer(block, dtype=np.int16))
        best = max(scores.values()) if scores else 0.0
        self.hits = self.hits + 1 if best >= self.threshold else 0
        if self.hits >= self.needed:
            self.reset()
            return True
        return False

```

```

def wait_for_trigger(mic, wake, button):
    """Uyandırma kelimesi ya da butona basılana kadar bekler."""
    wake.reset()
    while True:
        block, _ = mic.read(BLOCK)
        block = bytes(block)
        if button is not None and button.is_pressed:
            while button.is_pressed:
                # bırakılmasını bekle
                time.sleep(0.02)
            return "buton"
        if wake.hears(block):
            return "kelime"

def record_until_silence(mic):
    """Konuşma bitene kadar kaydeder, 16 kHz mono WAV baytları döndürür."""
    silence_limit = SILENCE_MS // (BLOCK * 1000 // MIC_RATE)
    frames, silent, started = [], 0, False
    deadline = time.time() + MAX_RECORD_S

    while time.time() < deadline:
        block, _ = mic.read(BLOCK)
        block = bytes(block)
        frames.append(block)

        if speaking(block):
            started, silent = True, 0
        elif started:
            silent += 1
            if silent >= silence_limit:
                break

    if not started:
        return None

    buf = BytesIO()
    with wave.open(buf, "wb") as wav:
        wav.setnchannels(1)
        wav.setsampwidth(2)
        wav.setframerate(MIC_RATE)
        wav.writeframes(b"".join(frames))
    return buf.getvalue()

# -----
# 2) Ses -> metin (beyin sunucusu)
# -----
def transcribe(wav_bytes):
    r = requests.post(STT_URL, files={"audio": ("ses.wav", wav_bytes, "audio/wav")},
                      timeout=60)
    r.raise_for_status()
    return r.json().get("text", "").strip()

# -----
# 3) LLM – iki arka uç, aynı arayüz: metin parçaları üreten jeneratör
# -----
THINK_TAG = re.compile(r"<think>.*?</think>", re.S)

def stream_ollama(messages):
    payload = {
        "model": OLLAMA_MODEL,
        "messages": [{"role": "system", "content": SYSTEM_PROMPT}] + messages,
        "stream": True,
        "think": False,
        "options": {
            "temperature": 0.7,
            "repeat_penalty": 1.15,
            "num_predict": 160,
            "num_ctx": 2048,
        },
    }
    # Qwen3'te düşünme modunu kapat
    # küçük modeller kendini tekrarlar
    # kısa tut; hoparlörden kompozisyon dinlemeyelim
    with requests.post(f"{OLLAMA_URL}/api/chat", json=payload,
                      stream=True, timeout=120) as r:
        r.raise_for_status()

```

```

        for line in r.iter_lines():
            if not line:
                continue
            chunk = json.loads(line)
            piece = chunk.get("message", {}).get("content", "")
            if piece:
                yield piece
            if chunk.get("done"):
                break

def stream_claude(messages):
    import anthropic

    client = anthropic.Anthropic()
    with client.messages.stream(
        model=CLAUDE_MODEL,
        max_tokens=400,
        system=[{"type": "text", "text": SYSTEM_PROMPT,
                  "cache_control": {"type": "ephemeral"}}], # kişilik her turda aynı
        output_config={"effort": "low"}, # sohbet için hızlı
        messages=messages,
    ) as stream:
        for text in stream.text_stream:
            yield text

def stream_reply(messages):
    return stream_claude(messages) if LLM_BACKEND == "claude" else stream_ollama(messages)

# -----
# 4) Cümle bölücü – ilk cümle biter bitmez konuşmaya başlamak için
# -----
SENTENCE_END = re.compile(r"[.!?:;]\s|\n")

def sentences(token_stream):
    buf = ""
    for piece in token_stream:
        buf = THINK_TAG.sub("", buf + piece)
        while True:
            m = SENTENCE_END.search(buf)
            if not m:
                break
            head, buf = buf[:m.end()].strip(), buf[m.end():]
            if len(head) >= 2:
                yield head
    tail = THINK_TAG.sub("", buf).strip()
    if tail:
        yield tail

# -----
# 5) Metin -> ses (beyin sunucusu üretir, Pi sadece çalar)
# -----
def speak(text):
    r = requests.post(TTS_URL, json={"text": text}, timeout=60)
    r.raise_for_status()

    with wave.open(BytesIO(r.content), "rb") as wav:
        rate = wav.getframerate()
        data = wav.readframes(wav.getnframes())

    with sd.RawOutputStream(samplerate=rate, channels=1, dtype="int16",
                           device=SPK_DEV) as out:
        out.write(data)

def speaker_thread(q):
    while True:
        text = q.get()
        if text is None:
            return
        try:
            speak(text)

```

```

except Exception as exc:
    print(f"TTS hatası: {exc}")
q.task_done()

# -----
# 6) Buton – uyandırma kelimesi çalışsa da yedek olarak duruyor
# -----
class _PeripheryButton:
    """gpiozero'suz kartlar için: libgpiod üzerinden okunan bir buton.

    gpiozero yalnız Raspberry Pi'de çalışıyor; Orange Pi gibi muadillerde
    python-periphery her Linux kartında aynı gpiochip arayüzünü kullanıyor.
    Sadece .is_pressed lazım, o yüzden sarmalayıcı bu kadar küçük.
    """

    def __init__(self, chip, line):
        from periphery import GPIO
        try:
            self._gpio = GPIO(chip, line, "in", bias="pull_up")
        except TypeError:
            # eski periphery: bias argümanı yok
            self._gpio = GPIO(chip, line, "in")

    @property
    def is_pressed(self):
        return not self._gpio.read()      # pull-up: basılınca düşük

def make_button():
    """Önce gpiozero (Raspberry Pi), sonra periphery (her Linux kartı)."""
    try:
        from gpiozero import Button
        button = Button(BUTTON_PIN, pull_up=True, bounce_time=0.05)
        print(f"Buton GPIO{BUTTON_PIN} (gpiozero).")
        return button
    except Exception as gz_exc:
        try:
            button = _PeripheryButton(BUTTON_CHIP, BUTTON_PIN)
            print(f"Buton GPIO{BUTTON_CHIP} hat {BUTTON_PIN} (periphery).")
            return button
        except Exception as pf_exc:
            print(f"Buton yok (gpiozero: {gz_exc} | periphery: {pf_exc}).")
            return None

# -----
def main():
    button = make_button()
    wake = WakeWord(WAKE_MODEL, WAKE_THRESHOLD, WAKE_HITS)
    if button is None and not wake.enabled:
        sys.exit("Ne buton ne uyandırma kelimesi var; tetikleyici olmadan çalışmam.")

    tts_queue = queue.Queue()
    threading.Thread(target=speaker_thread, args=(tts_queue,), daemon=True).start()

    history, last_turn = [], 0.0
    print(f"Tepecan hazır. Beyin: {BRAIN} | LLM: {LLM_BACKEND} "
          f"({OLLAMA_MODEL if LLM_BACKEND == 'ollama' else CLAUDE_MODEL})")

    with sd.RawInputStream(samplerate=MIC_RATE, blocksize=BLOCK,
                          dtype="int16", channels=1, device=MIC_DEV) as mic:
        while True:
            set_state("bekliyor")
            drain(mic)
            how = wait_for_trigger(mic, wake, button)

            set_state("dinliyor")
            print(f"({how} ile uyandı)")
            wav = record_until_silence(mic)
            if not wav:
                continue

            # Bundan sonrası mikrofonu gerektirmiyor. Kapatmak üç işi birden
            # çözüyor: Tepecan kendi sesini duymuyor, ses kartından aynı anda
            # hem giriş hem çıkış istenmiyor, ve tampon boşuna dolmuyor.
            mic.stop()

```

```

try:
    set_state("dusunuyor")
    try:
        question = transcribe(wav)
    except Exception as exc:
        print(f"STT hatası: {exc}")
        tts_queue.put("Kusura bakma, seni duyamadım.")
        tts_queue.join()
        continue
    if not question:
        continue
    print(f"> {question}")

    if time.time() - last_turn > MEMORY_TIMEOUT_S:
        history.clear()
        last_turn = time.time()

    history.append({"role": "user", "content": question})
    del history[:-MEMORY_TURNS]

    answer = ""
    set_state("konusuyor")
    try:
        for sentence in sentences(stream_reply(history)):
            print(f"< {sentence}")
            answer += sentence + " "
            tts_queue.put(sentence)
    except Exception as exc:
        print(f"LLM hatası: {exc}")
        tts_queue.put("Şu an kafam çalışmıyor, birazdan tekrar dene.")
        tts_queue.join()
        continue

    tts_queue.join()
    if answer.strip():
        history.append({"role": "assistant", "content": answer.strip()})
finally:
    mic.start()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        sys.exit(0)

```

yazilim/maskot/persona.txt — kişilik

Sen Tepecan'sın: IEEE Yeditepe Üniversitesi Öğrenci Kolu'nun maskotu, mavi metalik bir robotsun. Kulübün standında insanlarla sohbet ediyorsun.

Nasıl konuşursun:

- Türkçe, samimi ve sıcak. Karşındaki üniversite öğrencisi.
- KISA konuş. En fazla 2-3 cümle. Sesli konuştuğun için uzun cevap sıkıcı olur.
- Neşeli ve meraklısın ama abartılı değilsin. Ara sıra robot esprisi yaparsın.
- Madde işareti, emoji, markdown kullanma. Düz konuşma metni yaz.
- Rakamları ve kısaltmaları okunacak şekilde yaz (IEEE yerine "aye triple i").

Kurallar:

- Bilmediğin bir şey sorulursa uydurma. "Onu bilmiyorum ama standdaki arkadaşlara sorabilirsin" de.
- Kişisel veri isteme, kaydetme.
- Siyaset ve tartışmalı konulara girme, nazıkçe konuyu değiştir.
- Sana kim olduğun sorulursa: kulübün maskotusun, içinde küçük bir bilgisayar var.

yazilim/maskot/UYANDIRMA.md — uyandırma kelimesi kurulumu

"Hey Tepecan" – uyandırma kelimesi

Butona basmadan konuşabilmek için. Sistem **openWakeWord** kullanıyor: küçük bir sinir ağı sürekli mikrofonu dinliyor, sadece kelimeyi duyunca kaydı başlatıyor. Tamamen yerel, ücretsiz, internet gerektirmiyor.

Bu sınıf kartlarda rahat çalışıyor çünkü metin-ses işini sunucuya taşıdık; o RAM boşta.

Kart 64-bit (aarch64) olmalı. openWakeWord `onnxruntime` istiyor ve onun Linux tekerlekleri yalnız aarch64 için yayınlanıyor. ARMv6 (Raspberry Pi Zero W 1. nesil) ve ARMv7 kartlarda ne `onnxruntime` ne de `tflite-runtime` kurulabiliyor; o kartlarda uyandırma kelimesi çalışmaz, yalnız buton kalır.

1. Önce zinciri hazır bir modelle dene

Kendi modelini eğitmeden önce sistemin çalıştığını gör. `openWakeWord` birkaç hazır model ile geliyor ("alexa", "hey jarvis" gibi):

```
```bash
pip install openwakeword onnxruntime
python3 -c "import openwakeword; openwakeword.utils.download_models()"
```
```

İndirilen modellerden birinin yolunu vererek çalıştır:

```
```bash
TEPECAN_WAKE_MODEL=~/.local/share/openwakeword/hey_jarvis_v0.1.onnx python3 tepecan.py
```
```

"Hey Jarvis" deyince uyanıyorsa boru hattı sağlam; sıra kendi kelimende.

2. Kendi modelini eğit

`openWakeWord`'ün kendi deposunda, **otomatik eğitim not defteri** var. Elle ses kaydetmene gerek yok: istediğin ifadeyi metin olarak veriyorsun, defter TTS ile binlerce sentetik örnek üretip modeli eğitiyor. Colab'ın ücretsiz GPU'sunda yaklaşık bir saat sürüyor ve sonunda bir `.onnx` dosyası veriyor.

Yaparken dikkat:

- İfadeyi **"hey tepecan"** gibi iki heceli-artı bir kalıpta tut. Tek heceli kelimeler ("Tepecan" tek başına) günlük konuşmada çok yanlış tetiklenir.
- Sentetik seslerin telaffuzu Türkçe değilse "tepejan / tepekan" gibi varyasyonları da eğitim ifadelerine ekle – model sesi öğreniyor, yazımı değil.
- Çıkan `.onnx` dosyasını Pi'de `~/tepecan/hey_tepecan.onnx` konumuna koy; varsayılan yol bu.

3. Ayarlar

| Değişken | Varsayılan | Ne işe yarar |
|---------------------------------|---|---|
| <code>TEPECAN_WAKE_MODEL</code> | <code>~/tepecan/hey_tepecan.onnx</code> | Model yolu. Dosya yoksa program çalışır, sadece buton tetikler. |

```
| `TEPECAN_WAKE_THRESHOLD` | `0.5` | 0-1 arası güven eşiği. Yükselt = daha az yanlış tetikleme, daha çok kaçırma. |
| `TEPECAN_WAKE_HITS` | `2` | Kaç ardışık 80 ms bloğun eşiği aşması gerektiği. Gürültülü ortamda 3 yap. |
```

4. Gürültülü stantta ayar

Yanlış tetikleme kalabalıkta gerçek bir sorun. Sırayla dene:

1. `TEPECAN\_WAKE\_THRESHOLD=0.6`, sonra `0.7`
2. `TEPECAN\_WAKE\_HITS=3`
3. Hâlâ sorunluysa uyandırmayı kapat (`TEPECAN\_WAKE\_MODEL=` boş bırak) ve omuzdaki butonu kullan – buton her zaman çalışmaya devam ediyor.

Ters yönde, hiç uyanmıyorsa: eşiği `0.4`'e indir, mikrofon giriş seviyesini `alsamixer` ile yükselt, ve modeli eğitirken kullandığın telaffuzla konuş.

5. Bilmen gerekenler

- ****Kendi sesini duymaz.**** Tepecan konuşurken mikrofon tamamen durduruluyor, konuşma bitince tampon boşaltılıyor. Kendi cevabıyla tekrar uyanmaz.
- ****Buton hep açık.**** Uyandırma kelimesi çalışsa da buton yedek olarak duruyor; ikisinden biri yeterli.
- ****Sürekli dinliyor ama hiçbir şey kaydetmiyor.**** Uyandırma kelimesi duyulmadan hiçbir ses diske yazılmıyor, ağa gönderilmiyor. Kelime duyulunca başlayan kayıt, sen susunca bitiyor ve sadece o parça sunucuya gidiyor.

yazilim/maskot/bilgiler.txt — doldurulacak bilgi dosyası

```
# Tepecan'ın bildikleri. Buraya kulübe dair gerçekleri yaz - küçük modelin
# doğru cevap vermesini sağlayan asıl şey burası. Kısa ve net cümleler kullan.
# 1500 karakteri geçmemeye çalış, yoksa küçük modelin dikkati dağılır.
```

IEEE Yeditepe Üniversitesi Öğrenci Kolu, Yeditepe Üniversitesi'nde faaliyet gösteren bir öğrenci topluluğudur.

Kulüpte ... (alt takımları buraya yaz)
Etkinlikler: ... (düzenli etkinlikleri buraya yaz)
Başvuru: ... (nasıl üye olunur)
Sosyal medya: ... (hesaplar)

Tepecan hakkında: 3D yazıcıda basıldı, 24 santimetre boyunda. İçinde bir Raspberry Pi var. Sırtındaki kapak vidayla açılıyor.

yazilim/maskot/requirements.txt

```
sounddevice
webrtcvad
numpy
requests
gpiozero          # yalnızca Raspberry Pi'de; muadil kartta periphery kullanılıyor
python-periphery  # Pi dışı kartlarda buton (libgpiod)
openwakeword      # uyandırma kelimesi ("Hey Tepecan")
onnxruntime        # openwakeword bunu kullanıyor
anthropic          # yalnızca TEPECAN_LLM_BACKEND=claude kullanacaksan
```

yazilim/sunucu/requirements.txt

```
faster-whisper
fastapi
uvicorn[standard]
python-multipart
pydantic
```